

COSMIC Measurement Manual for ISO 19761

Part 1: Principles, Definitions & Rules

Definitions

control command

command that enables human functional users to control their use of the software but which does not involve any movement of data about an object of interest of the FUR of the software being measured.

data attribute

smallest parcel of information, within an identified data group, carrying a meaning from the perspective of the software's Functional User Requirements.

data group / data group type

distinct, non-empty, non-ordered and non redundant set of data attributes where each included data attribute describes a complementary aspect of the same one object of interest.

Entry type

data movement that moves a data group from a functional user across the boundary into the functional process where it is required.

Exit type

data movement that moves a data group from a functional process across the boundary to the functional user that requires it.

event

something that happens.

functional process type

elementary component of a set of Functional User Requirements, comprising a unique, cohesive and independently executable set of data movements.

functional user

user that is a sender and/or an intended recipient of data in the Functional User Requirements of a piece of software.

FUR

sub-set of the user requirements describing what the software shall do, in terms of tasks and services.

level of decomposition

any level resulting from dividing a piece of software into components (named "Level 1", for example), then from dividing components into sub-components ("Level 2"), then from dividing sub-components into sub-sub components ("Level 3"), etc.

level of granularity

any level of expansion of the description of any part of a single piece of software (e.g. a statement of its requirements, or a description of the structure of the piece of software) such that at each increased level of expansion, the description of the functionality of the piece of software is at an increased and uniform level of detail.

OOI - Abbreviation for object of interest

object of interest / object of interest type

any “thing” that is identified from the point of view of the Functional User Requirements about which the software is required to process and/or store data.

persistent storage

storage which enables a functional process to store data beyond the life of the functional process and/or which enables a functional process to retrieve data stored by another functional process, or stored by an earlier occurrence of the same functional process, or stored by some other process.

Read type

data movement that moves a data group from persistent storage within reach of the functional process which requires it.

sub-process type

part of a functional process that either moves data (into the software from a functional user or out of the software to a functional user, or to or from persistent storage) or that manipulates data.

system

combination of hardware, software and manual procedures organized to achieve stated purposes.

triggering Entry type

the Entry data movement of a functional process that moves a data group generated by a functional user that the functional process needs to start processing.

triggering event type

event that causes a functional user of the piece of software to initiate (“trigger”) one or more functional processes.

user

any person or thing that communicates or interacts with the software at any time.

Write type

data movement that moves a data group from a functional process to persistent storage.

Identification Rules

RULE 3: Identification of the FUR

The FUR identified to be within the scope of the FSM shall be used as the exclusive source from which the functional size of the software is to be measured.

NOTE: The term “FUR” means those functional user requirements that are completely defined so that a COSMIC functional size measurement is possible.

RULE 7: Functional Users

All functional users that trigger, provide information to, or receive information from functional processes in the FUR of the software within the scope of the FSM shall be identified.

NOTE 1: This rule above corrects an omission in ISO 19761. An amendment to the standard is in preparation.

NOTE 2: As persistent storage is on the software side of the boundary, it is not considered to be a functional user of the software being measured.

RULE - Identification of boundaries

- RULE 8: The boundary of each piece of software within each layer and in the scope of the FSM shall be identified.
- RULE 9: Once the boundaries have been identified, each FUR within the scope of the FSM shall be allocated to a piece of software.

Functional Processes and Data Groups

RULE 10: Identification of functional processes

Each functional process identified in the scope of the FSM shall:

- a) be derived from at least one identifiable FUR,
- b) be initiated by an Entry data movement from a functional user informing the functional process that it has detected a triggering event,
- c) comprise at least two data movements, namely always one Entry plus either an Exit or a Write.
- d) belongs to one, and only one layer,
- e) be complete when a point of asynchronous timing is required to be reached according to its FUR.

NOTE 1: The COSMIC Group has subsequently clarified sub-clause e) above as the equivalent of the following statement: “the set of all data movements that is needed to meet its FUR for all the possible responses to its triggering Entry”.

NOTE 2: The Generic Software Model is a logical model. A functional process occurrence may start processing before data has been entered e.g. when a human user clicks on a menu to display a blank screen for data entry.

NOTE 3: In a set of FUR, each event which causes a functional user to trigger a functional process cannot be sub-divided for that set of FUR, and has either happened or it has not happened.

RULE 11: Identification of objects of interest and data groups

Each data group identified in the scope of the FSM shall:

- a) be unique and distinguishable through its unique collection of data attributes,
- b) be directly related to one object of interest described in the software's FUR.

NOTE 1: An object of interest can be any physical thing, as well as any conceptual thing or part of a conceptual thing in the world of a functional user.

NOTE 2: Examples of 'thing' include, but are not limited to, software applications, humans, sensors, or other hardware.

NOTE 3: The term object of interest is used in order to avoid terms related to specific software engineering methods. The term does not imply objects in the sense used in Object Oriented methods. Similarly, the word Entity is avoided because of its use in Data Modelling.

NOTE 4: Constants or variables which are internal to the functional process, or intermediate results in a calculation, or data stored by a functional process resulting only from the implementation, rather than the FUR, are not data groups.

RULE 12: Identification of data movements

Each functional process identified in 4.2 shall be partitioned into its component data movements.

NOTE: The COSMIC method defines a data movement type as a BFC.

Data Movement Counting Rules

RULE 13: Functional Process - Single Entry

For any one functional process, a single Entry data movement shall be identified and counted for the entry of all data describing a single object of interest that the FUR require to be entered, unless the FUR explicitly require data describing the same single object of interest to be entered more than once in the same functional process.

RULE 14: Functional Process - Single Exit, Read or Write

Similarly, a single Exit, Read or Write data movement shall be identified and counted for the movement of all data describing a single object of interest that the FUR requires of that type (e.g. Exit, Read or Write, respectively), unless the FUR explicitly require data describing the same single object of interest to be moved more than once in the same functional process by a data movement of the same type (e.g. Exit, Read or Write, respectively).

RULE 15: Functional Process - Occurrences

If a data movement of a particular type (Entry, Exit, Read or Write) occurs multiple times with different data values when a functional process is executed, only one data movement of that type shall be identified and counted in that functional process.

Classification of Data Movements

RULE 16: Entry

An Entry shall:

- a) receive a single data group which originates from the functional user side of the boundary,

- b) account for all required formatting and presentation manipulations along with all associated validations of the entered data attributes, to the extent that these data manipulations do not involve another type of data movement.
- c) include any “request to receive the Entry data” functionality, where it is unnecessary to specify what data should be entered.

NOTE: An Entry accounts for all manipulations that might be required to validate some entered codes or to obtain some associated descriptions. However, if one or more Reads are required as part of the validation process, these are identified and counted as separate Read data movements.

RULE 17: Exit

An Exit shall:

- a) send data attributes from a single data group to the functional user side of the boundary,
- b) account for all required data formatting and presentation manipulations, including processing required to send the data attributes to the functional user, to the extent that these manipulations do not involve another type of data movement.

RULE 18: Read

A Read shall:

- a) retrieve a single data group from persistent storage.
- b) account for all logical processing and/or mathematical computation needed to read the data, to the extent that these manipulations do not involve another type of data movement,
- c) include any “request to read” functionality.

RULE 19: Write

A Write shall:

- a) move data attributes from a single data group to persistent storage.
- b) account for all logical processing and/or mathematical computation to create the data attributes to be written, to the extent that these manipulations do not involve another type of data movement.

RULE 20: Write - Delete

A requirement to delete a data group from persistent storage shall be a single Write data movement.